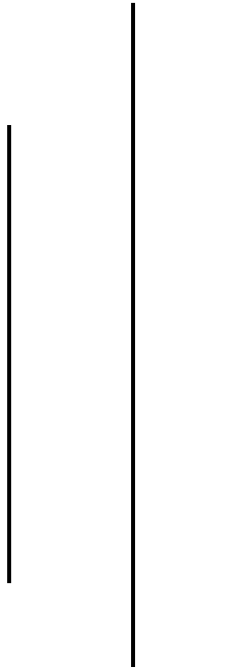


NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University



LAB REPORT FOR EMBEDDED SYSTEMS



LAB REPORT 5: VHDL CODE FOR SUBTRACTOR

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20 th Nov	
2.	VHDL Code (OR & NOT Gate) using testbench	23 rd Nov	
3.	Universal Gates and Special Gates	30 th Nov	
4.	VHDL Code for Adder	7 th Dec	
5.	VHDL Code for Subtractor	14 th Dec	

LAB 5: VHDL CODE FOR SUBTRACTOR

OBJECTIVE:

The objective of this lab is to understand the basic concept of binary subtraction by designing and implementing a Half Subtractor circuit using VHDL. This lab aims to familiarize students with combinational logic design, Boolean expressions of subtractors, and VHDL coding for digital subtraction circuits. It also helps in simulating and verifying the correctness of subtractor outputs using different input combinations.

THEORY:

Introduction to Subtractor

A subtractor is a combinational digital circuit that performs subtraction of binary numbers. It is an essential component in arithmetic logic units (ALUs), processors, and digital systems where arithmetic operations are required. Depending on the number of input bits and borrow handling capability, subtractors are classified into Half Subtractor and Full Subtractor.

Half Subtractor

A Half Subtractor is a combinational logic circuit that subtracts one single-bit binary number from another single-bit binary number. It produces two outputs:

- **Difference (D)**
- **Borrow (B)**

It does not consider any borrow from a previous stage, hence it is called a half subtractor.

Inputs

- A (Minuend)
- B (Subtrahend)

Outputs

- Difference (D)
- Borrow (B)

Logic Equations

- Difference (D) = $A \oplus B$
- Borrow (B) = $\bar{A} \cdot B$

Truth Table

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

The difference output represents the result of subtraction, while the borrow output indicates whether a borrow is required from the higher significant bit.

Half subtractors are simple circuits but cannot be used alone for multi-bit subtraction because they do not handle borrow input from previous stages.

Full Subtractor

A Full Subtractor subtracts two single-bit binary numbers while also considering a borrow input from the previous less significant stage. It produces two outputs:

- **Difference (D)**
- **Borrow Out (Bout)**

Inputs

- A (Minuend)
- B (Subtrahend)
- Bin (Borrow input)

Outputs

- Difference (D)
- Borrow Out (Bout)

Logic Equations

- Difference (D) = $A \oplus B \oplus \text{Bin}$
- Borrow Out (Bout) = $(\bar{A} \cdot B) + (B \cdot \text{Bin}) + (\bar{A} \cdot \text{Bin})$

Truth Table

A	B	Bin	Difference	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

A full subtractor can be implemented using two half subtractors and an OR gate. The first half subtractor subtracts B from A, and the second half subtractor subtracts the borrow input from the difference of the first stage. The borrow outputs are then combined using an OR gate to generate the final borrow output.

Subtractor Implementation Using VHDL

VHDL (VHSIC Hardware Description Language) is used to describe the behavior and structure of digital circuits. In this lab, VHDL is used to model full subtractor circuits using concurrent signal assignments and Boolean logic expressions. Simulation of the VHDL code helps in verifying the correct operation of the subtractor for all possible input combinations.

DATAFLOW FOR HALF SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity arpan_halfsub is
    port (
        A : in std_logic;
        B : in std_logic;
        diff : out std_logic;
        borrow : out std_logic
    );
end arpan_halfsub;
architecture hs_arch of arpan_halfsub is
begin
    process
    begin
        diff <= A xor B;
        borrow <= (not A) and B;
    end process;
end hs_arch;
```

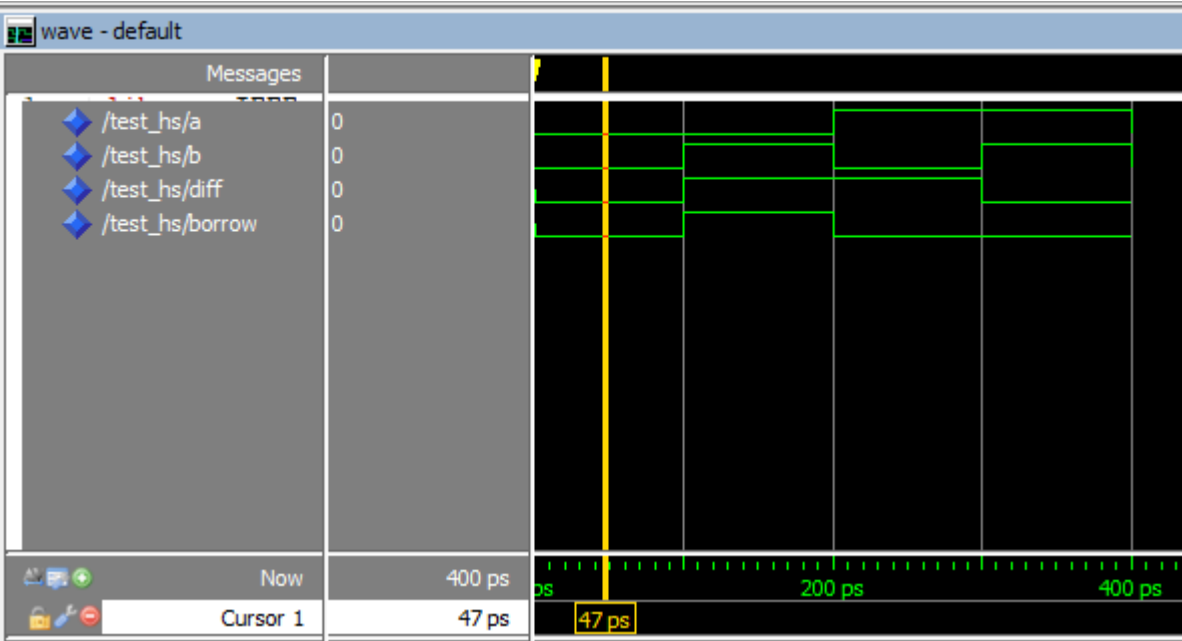
BEHAVIORAL FOR HALF SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity arpan_halfsub is
    port (
        A : in std_logic;
        B : in std_logic;
        diff : out std_logic;
        borrow : out std_logic
    );
end arpan_halfsub;
architecture hs_arch of arpan_halfsub is
begin
    process (A,B)
    begin
        if (A = B) then
            diff <= '0'; borrow <= '0';
        elsif (A = '0' and B = '1') then
            diff <= '1'; borrow <= '1';
        else
            diff <= '1'; borrow <= '0';
        end if;
    end process;
end hs_arch;
```

TESTBENCH FOR HALF SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;
entity test_hs is
end test_hs;
architecture test_arch of test_hs is
component arpan_halfsub
    port(
        A,B : in std_logic;
        diff, borrow : out std_logic
    );
end component;
signal A,B : std_logic := '0';
signal diff, borrow : std_logic;
begin
UUT: arpan_halfsub port map (
    A => A,
    B => B,
    diff => diff,
    borrow => borrow
);
stim_proc : process
begin
    A <= '0'; B <= '0';
    wait for 100 ps;
    A <= '0'; B <= '1';
    wait for 100 ps;
    A <= '1'; B <= '0';
    wait for 100 ps;
    A <= '1'; B <= '1';
    wait for 100 ps;
end process;
end test_arch;
```

WAVEFORM FOR HALF SUBTRACTOR:



DATAFLOW FOR FULL SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity arpan_fullsub is
    port (
        A : in std_logic;
        B : in std_logic;
        Bin : in std_logic;
        diff : out std_logic;
        borrow : out std_logic
    );
end arpan_fullsub;
architecture hs_arch of arpan_fullsub is
begin
    process
    begin
        diff <= A xor B xor Bin;
        borrow <= (not A and B) or (not A and Bin) or (B and Bin);
    end process;
end hs_arch;
```

BEHAVIORAL FOR FULL SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity arpan_fullsub is
    port (
        A : in std_logic;
        B : in std_logic;
        C : in std_logic;
        diff : out std_logic;
        borrow : out std_logic
    );
end arpan_fullsub;
architecture hs_arch of arpan_fullsub is
begin
    process (A,B,C)
    begin
        if (A = '0' and B = '0' and C = '0')
            or (A = '1' and B = '0' and C = '1')
            or (A = '1' and B = '1' and C = '0') then
            diff <= '0'; borrow <= '0';
        elsif (A = '0' and B = '0' and C = '1')
            or (A = '0' and B = '1' and C = '0')
            or (A = '1' and B = '1' and C = '1') then
            diff <= '1'; borrow <= '1';
        elsif (A = '0' and B = '1' and C = '1')
            diff <= '0'; borrow <= '1';
        else (A = '1' and B = '0' and C = '0')
            diff <= '1'; borrow <= '0';
        end if;
    end process;
end hs_arch;
```

TESTBENCH FOR FULL SUBTRACTOR:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity test_hs is
end test_hs;

architecture test_arch of test_hs is
component arpan_fullsub
  port(
    port (
      A : in std_logic;
      B : in std_logic;
      Bin : in std_logic;
      diff : out std_logic;
      borrow : out std_logic
    );
end component;

signal A,B, Bin : std_logic :='0';
signal diff, borrow : std_logic;

begin
UUT: arpan_halfsub port map (
  A => A,
  B => B,
  Bin => Bin,
  diff => diff,
  borrow => borrow
);
```

```
stim_proc : process
begin
  a <= '0' ; b<='0';bin<='0';
  wait for 100ps;
  a <= '0' ; b<='0';bin<='1';
  wait for 100ps;
  a <= '0' ; b<='1';bin<='0';
  wait for 100ps;
  a <= '0' ; b<='1';bin<='1';
  wait for 100ps;
  a <= '1' ; b<='0';bin<='0';
  wait for 100ps;
  a <= '1' ; b<='0';bin<='1';
  wait for 100ps;
  a <= '1' ; b<='1';bin<='0';
  wait for 100ps;
  a <= '1' ; b<='1';bin<='1';
  wait for 100ps;
end process;
end test_arch;
```

RESULT:

The Half Subtractor circuit was successfully designed and implemented using VHDL. The simulation was carried out for all possible input combinations of A and B. The obtained Difference and Borrow outputs were found to be correct and matched the theoretical truth table of the half subtractor. The simulation waveforms clearly showed that the Difference output follows the XOR operation of the inputs, while the Borrow output is generated when the subtrahend is greater than the minuend.

CONCLUSION:

In this experiment, the Half Subtractor circuit was successfully implemented and verified using VHDL. The relationship between binary subtraction and combinational logic was clearly understood. The results obtained from simulation confirm the correct operation of the half subtractor. This experiment provides a foundation for understanding more complex subtraction circuits such as the Full Subtractor and multi-bit binary subtractors.